Defect Prevention Process

Doc ID: HH-CMM-04 **Version:** 1.0 **Effective Date:** 2026-08-29 **Owner:** Venkat Narasimha (Processbricks) **Review Cycle:** Annual (next: 2027-08-29) **Last Reviewed:** 2026-08-29

> **Classification:** Internal **One-line summary:** We commit to finding the *root cause* of every SEV-1/2, not just the symptom — and to writing at least one lesson per incident so the same defect doesn't happen twice.

1. Purpose

Defects are inevitable. What separates a Level-5 organisation from a Level-3 one is not "no defects" — it's "we change the process so the defect class disappears, not just this defect instance".

LEARNINGS.md is our defect-prevention repository. Every post-mortem feeds it; every LEARNINGS entry feeds a runbook. Without this loop, the same defect returns every six months with a different face (LEARNINGS #113/#114 backup streak; LEARNINGS #153 gunicorn restart; the recurring "did the cron actually run?" question). With this loop, every defect we encounter is the last defect of its kind — or at least, the *next* one is caught faster.

This document codifies:

- **RCA techniques** — 5 Whys, Fishbone, Pareto, Fault Tree — when to use which.
- **Causal-analysis meeting structure** — who, when, agenda.
- **Lessons-learned repository** — when to write, what format, review cadence.
- **Defect categorisation** — by type, severity, detection point.
- **Examples from Haritha** — what we got right, what we got wrong, what we dismissed.
- **Defect prevention metrics** — defect density, recurrence rate, time-to-detection.
- **Integration with PDCA** — defect prevention IS the Act step of 10-process-improvement.

The hardest part of defect prevention is **listening when the subagent's diagnosis is right but your hypothesis says otherwise**. LEARNINGS #72 (verify-before-acting) and the CapitalCase color fix (dismissed-correct-diagnosis) are the canonical case studies here.

2. Scope

2.1 In scope

- **RCA techniques** — 4 canonical methods with Haritha-specific guidance.
- **Causal-analysis meeting** — agenda, participants, output.
- **Lessons-learned loop** — LEARNINGS.md as the repo; format; review cadence.
- **Defect categorisation** — by type, severity, detection point.
- **Defect prevention metrics** — definitions, sources.
- **Integration with PDCA** — defect prevention as the Act step.
- **Examples from Haritha history** — what worked, what didn't.

2.2 Out of scope

- **Incident response itself** — see 07-incident-management. Defect prevention assumes the incident is over; it's about preventing recurrence.
- **Change management approval** — see 05-process/05.1-change-management. The defect prevention output is a proposed change; the change still goes through approval.
- **Post-mortem template** — see 05.2. This doc defines the *analysis* part; 05.2 is the *write-up* template.

3. Policy Statement

3.1 What we commit to

Haritha Hospitals commits to:

1. **Every SEV-1 and SEV-2 incident has a post-mortem within 24h** (07 §3.6).
2. **Every post-mortem does at least one RCA pass** using a method from §3a (typically 5 Whys; Fishbone for multi-cause; Pareto for "what's the top contributor?").
3. **Every post-mortem writes at least one LEARNINGS.md entry** — numbered, cited in runbooks, post-mortem references the lesson number.
4. **Every LEARNINGS.md entry is reviewed monthly** for accuracy and citation completeness.
5. **A causal-analysis meeting is convened within 48h of any SEV-1** — agenda per §3b. SEV-2 meetings are optional but recommended.
6. **Defects are categorised** by type + severity + detection point within 7 days of the post-mortem landing.
7. **Recurrence rate is tracked** — if the same root cause reappears within 90 days, that's a "policy violation" classification (07 §6a Edge 3) and triggers a fresh post-mortem.
8. **The Act step of every PDCA cycle includes a defect-prevention check** — does this cycle prevent a class of defects, or just one instance?

3.2 RCA techniques (canonical)

We use four techniques. Each has a specific use case.

3.2.1 Five Whys

- **What it is.** Iteratively ask "why?" five times (or until the answer stops being actionable). Originated at Toyota in the 1950s.
- **Best for.** Single-cause issues with linear causation.
- **Example (LEARNINGS #113):**
  1. Why did the backup script fail? — `ls *.tar.gz` returned nothing.
  2. Why did `ls` return nothing? — Glob matched no files.
  3. Why did no tarball exist? — Script never created one; it only copied 4 loose files.
  4. Why did the script never create a tarball? — Original design assumed the 4 files were already tarred upstream; they weren't.
  5. Why did the assumption go unchallenged? — No integration test verified the tarball existed before the rsync step.
- **Root cause.** Missing test for pre-condition.

3.2.2 Fishbone (Ishikawa)

- **What it is.** Categorise potential causes across 6 categories: **Method, Machine, Material, Manpower, Measurement, Environment**. Originated at Kawasaki in the 1960s.
- **Best for.** Multi-cause issues where the 5-Whys linear chain doesn't capture the breadth.
- **Example (LEARNINGS #153, gunicorn 500-outage):**
  - **Method** — `bench install-app` doesn't restart gunicorn; deploy runbook didn't include restart step.
  - **Machine** — Gunicorn `--preload` freezes sys.path; container design amplifies the issue.
  - **Material** — `.pth` file written but invisible to frozen sys.path.
  - **Manpower** — No "after install-app, smoke test" habit; install looked successful.
  - **Measurement** — Heartbeat probed "is container up?" not "is backend responsive to all routes?".

  - **Environment** — Dev container restarted 9d ago; prod 2d ago; both with stale sys.path; install triggered regression on both simultaneously.
- **Root cause.** Combination of (Method) missing step + (Machine) gunicorn design + (Measurement) insufficient smoke test. Fix addresses all three.

3.2.3 Pareto

- **What it is.** 80/20 rule. Identify which ~20% of causes produce ~80% of defects. Often visualised as a bar chart sorted descending.
- **Best for.** Prioritisation — "what's the top contributor to our incidents?"
- **Example (LEARNITH archive).** Pareto of LEARNINGS by category:
  - Infra/deploy issues: ~35% of all lessons.
  - Schema/migration: ~25%.
  - Backup/recovery: ~15%.
  - Auth/access: ~10%.
  - Other: ~15%.
- **Action.** Most PDCA cycles should target the infra/deploy category — that's where the leverage is.

3.2.4 Fault Tree Analysis

- **What it is.** Top-down deductive analysis. Start with the undesired event; decompose into combinations of lower-level events using AND/OR gates. Originated at Bell Labs in the 1960s for aerospace.
- **Best for.** Complex system failures where multiple subsystems must fail simultaneously.
- **When to use at Haritha.** Rare. Reserved for "this would only happen if A AND B AND C all went wrong" scenarios. Example: the 08 §6a Edge 9 total-loss scenario would benefit from a fault tree — main VPS down AND offsite unreachable AND no cold storage.
- **Trade-off.** High effort; only use when simpler methods don't capture the complexity.

3.3 Causal-analysis meeting structure

Within 48h of a SEV-1, convene a causal-analysis meeting.

Participants:

- Venkat (owner, decision-maker).
- Admin who led the response (or, if Venkat led it, the subagent that diagnosed).
- Anyone else materially involved (e.g., user who reported the original symptom).

Agenda (45 min target):

1. **Timeline review (10 min).** Walk through the post-mortem timeline. Confirm accuracy.
2. **Hypothesis review (10 min).** What was the working hypothesis at each phase? When did it change? Why?
3. **5 Whys (10 min).** Apply 5 Whys to the confirmed root cause. Document the chain.
4. **Fishbone (5 min, optional).** If multi-cause, categorise across the 6 categories.
5. **Lessons (5 min).** Identify the LEARNINGS entry/entries. Draft titles.
6. **Action items (5 min).** What changes (runbook, policy, code) prevent recurrence? Owners, due dates.

Output:

- Post-mortem file with the RCA section filled in.
- LEARNINGS.md entries drafted (numbered, ready to commit).
- Action items tracked in 07 §9 implementation checklist.

3.4 Lessons-learned repository — LEARNINGS.md

- **When to add.** Every post-mortem. Every significant change (deploy that touches critical code path). Every subagent that surfaces a non-obvious finding.
- **Format.**

```
### Lesson #N: <one-line title> (<YYYY-MM-DD>)

**What happened:** <1-3 sentences, the symptom>.
**Why it happened:** <root cause>.
**Fix / lesson:** <what we did or should do>.
**Cross-refs:** <runbooks, policies, post-mortems>.
```

- **Numbering.** Sequential. Existing numbers preserved (LEARNINGS #1-#157). New lessons continue the series (next would be #158). The numbers are NOT version numbers; they are stable identifiers.
- **Review cadence.** Monthly (PA reviews new entries, fixes typos, ensures cross-refs are valid).
- **Anti-pattern.** "Lessons that don't get applied" — a lesson no runbook or policy cites is a lesson no one will see. (§7 of every policy lists lessons cited; that list is the assertion the lesson is applied.)

3.5 Defect categorisation

Every defect (i.e., every LEARNINGS entry or post-mortem-rooted finding) gets three tags.

3.5.1 By type

Type	Definition	Examples
schema	Database schema, migration, or fixtures issue	LEARNINGS #152 (module vs app column), #155 (Letter Head no module)
infra	Docker, container, network, volume, DNS, cert	LEARNINGS #153 (gunicorn –preload), #89 (apps.txt drift)
code	Bug in custom app code or hook	(no canonical example yet — slot exists)
data	Migration script, master data, lookup issue	LEARNINGS #157 (migration idempotency)
process	Runbook, policy, workflow, or process gap	LEARNINGS #72 (verify-before-acting), #90 (heartbeat freshness)

3.5.2 By severity

Severity	Definition
critical	SEV-1 incident, data loss, security breach
major	SEV-2 incident, repeated near-miss
minor	SEV-3, isolated bug, one-time surprise
cosmetic	SEV-4, naming, alignment, copy

3.5.3 By detection point

Detection	Definition	Examples
dev	Caught before any other env saw it	(rare at Haritha; small dev)
staging	Caught in QA before prod	(no QA-only catches yet; rare)
prod	First saw it in prod	LEARNINGS #113, #153, #154
user-reported	A user reported before we noticed	LEARNINGS #88 (scheduler.log)

The detection-point tag drives the "test in lower env first" question. A prod-only detection usually means our dev/QA didn't exercise the same code path.

3.6 Defect prevention metrics

- **Defect density.** Defects per change (deploy or LEARNINGS entry) over a window. Source: LEARNINGS.md cross-ref git log.
- **Recurrence rate.** % of SEV-1/2 incidents whose root cause matches a prior lesson within 90 days. Source: post-mortem root-cause vs LEARNINGS text search. Target: < 5%.
- **Time-to-detection.** Time from defect introduced to defect detected. For infra/deploy: time from deploy to first failure report. Source: git log + post-mortem timeline.
- **Time-to-RCA.** Time from incident detection to root-cause confirmation. Source: post-mortem timeline. Target: < 30 min for SEV-1.
- **Lessons-citation coverage.** % of LEARNINGS entries cited in at least one runbook or policy. Target: 100%.

3a. Current State (as of 2026-08-29)

3a.1 What we have TODAY

Component	Where it lives	Status
Post-mortem template	../05-process/05.2	Live
Post-mortem example (2026-08-29)	../05-process/05.2 §"Part 2"	Live
LEARNINGS.md (157 entries)	../../../../.learnings/LEARNINGS.md	Live
Lessons-cited footers in runbooks + policies	this folder, ../04-runbooks	Live (mostly; gaps exist)
5 Whys in post-mortem template	implicit (template has "Root cause" + "Why wasn't this caught earlier?")	Live
Fishbone analysis	not formally used	**Not Started**
Pareto analysis	not formally used	**Not Started**
Causal-analysis meeting structure	this doc §3.3	New (codified today)
Defect categorisation	not enforced	**Not Started**
Recurrence rate tracking	not measured	**Not Started**
Defect density metric	not measured	**Not Started**

3a.2 What is WORKING

- **LEARNINGS.md as a repository.** 157+ entries, all numbered, all with cross-refs to other lessons, runbooks, and policies. This is the spine of our defect-prevention practice.
- **Post-mortem → lesson → runbook loop.** The 2026-08-29 outage produced LEARNINGS #153, #154 within 4h; runbooks 04.1, 04.4, 05.1, 05.2 updated same day. Closed loop.
- **"Lessons cited" in policy footers.** 07-incident-management §7 lists LEARNINGS citations explicitly. Other policies follow.
- **Blameless language.** Post-mortems focus on what happened, not who did what. The "CapitalCase color fix mea culpa" entry in memory/2026-08-28.md is honest without being blaming.

3a.3 Known GAPS

1. **No formal Fishbone.** Multi-cause incidents get prose analysis, not structured 6-category decomposition. Future improvement: add Fishbone template to 05.2.
2. **No Pareto.** We don't know which category of defect (schema / infra / code / data / process) dominates. Future improvement: monthly Pareto review.
3. **No causal-analysis meeting cadence.** 48h-from-SEV-1 is policy now; we haven't held one (the 2026-08-29 outage was handled in chat). Future improvement: first formal meeting.
4. **Defects not categorised.** LEARNINGS.md entries have type + severity + detection tags only informally. Future improvement: tagging convention in §3.5.
5. **Recurrence rate unmeasured.** Have we ever had the same defect class twice? Yes (cron-recovery + silent-failures are related). But we don't measure.
6. **No "near-miss" log.** We capture incidents; we don't capture "almost-incidents" that were caught by verification (LEARNINGS #72 in action). Future improvement: near-miss entries in LEARNINGS.md prefixed `#NM-`.
7. **Defect prevention metrics not wired to QPM.** 11 §3a doesn't list recurrence rate or defect density. Future: add to M-list.

These gaps are explicit v1 scope decisions. Listing them is transparency, not apology.

3b. Concrete Examples (Haritha history)

Example 1 — LEARNINGS #113 / #114 (backup silent failure) — 5 Whys + PDCA applied

Already documented in 10-process-improvement §3b Example 1. What makes it a defect-prevention case study: the LEARNINGS entry doesn't just say "fix the script"; it says "never rely on `ls` for 'does this glob match?'" — a general principle that prevents the entire class of bug.

RCA applied: 5 Whys (LEARNINGS #113 has the chain). **Defect class:** process (silent-failure class). Recurrence risk: medium. (LEARNINGS #110 jq-silent-zero is the same class — caught by the same principle.) **Cited in runbooks:** 04.3 §"Backup verification"; 05-operations-security §3.1.

Example 2 — LEARNINGS #153 (gunicorn `--preload` outage) — Fishbone applied

Already documented in 07 §3b Example 1 + §3b Fishbone example above.

RCA applied: 5 Whys (single chain — gunicorn freezes sys.path) + Fishbone (multi-cause decomposition across 6 categories). **Defect class:** infra (container/gunicorn design) + process (runbook gap). **Fix scope:** Add `docker restart` to deploy runbook (04.1) — addresses the

Method category. Heartbeat probe could add a smoke-test after install — addresses the Measurement category. The Machine category is upstream (Frappe design); we mitigate, don't fix.

Example 3 — LEARNINGS #154 (DB password drift) — single defect, blunt fix

- **What.** Hardcoded credential in MEMORY.md drifted from container env. 401 on correct password.
- **RCA.** 5 Whys:
  1. Why 401? — Password didn't match.
  2. Why didn't it match? — MEMORY.md had old value.
  3. Why was old value still in MEMORY.md? — Rotation in a prior session updated env but not MEMORY.md.
  4. Why is credential hardcoded in a doc? — Process gap; should always read from container.
  5. Why doesn't the process enforce read-from-container? — No rule.
- **Root cause.** No rule "always read credential from container env".
- **Fix.** Add the canonical read pattern (`docker exec erp-prod-db-1 printenv MYSQL_ROOT_PASSWORD`) to incident response. Add `CAUTION: literals may be stale` banner to MEMORY.md.
- **Defect class.** process.
- **Cited in runbooks:** 04.4 §"Login 401"; 07 §3b Example 3.

Example 4 — Roster crash (CapitalCase colors) — dismissed correct diagnosis (ANTI-EXAMPLE)

Already referenced in 09 §3b Example 7. Three fix attempts; two wrong; one right.

Why this is a defect-prevention anti-example:

- The subagent's correct diagnosis ("submit draft SAs") was dismissed by Venkat for a wrong reason ("SAs look fine").
- The actual root cause (CapitalCase vs lowercase Tailwind names) was found only after three attempts.
- The LEARNINGS entry came after the fact; no causal-analysis meeting was held.
- The lesson is logged in memory/2026-08-28.md as "I blamed HRMS framework when root cause was my own Phase 4.1 hex color palette. Should have read JS source first per Lesson #140."

The defect-prevention principle violated: LEARNINGS #72 — verify before acting. Venkat dismissed the subagent without verifying the subagent's claim against the JS source. The fix would have taken one extra minute; instead it took three cycles and a mea culpa.

Defect class. process (verification gap). **Cited where.** memory/2026-08-28.md + this doc.

Example 5 — LEARNINGS #90 (heartbeat carry-forward) — Pareto candidate

The heartbeat reported 77% disk carried forward from Aug 20 06:00 IST for 26 hours. Actual current state: 95% / 4.1G free.

RCA. 5 Whys: 1. Why was the report wrong? — Stale data carried forward. 2. Why stale? — Probe didn't run on schedule. 3. Why didn't probe run? — Subagent session limit hit. 4. Why isn't there a cron-based fallback? — We assumed heartbeat subagent would always be available. 5. Why that assumption? — Design assumption from when we had only one operator/subagent setup.

Root cause. Single point of failure in heartbeat (subagent only, no cron fallback).

Fix. Add rule "fresh probe every ≤ 24h for drift-prone metrics" + cron-based heartbeat shell script as authoritative fallback.

Defect class. process + infra (cron fallback).

Why this is a Pareto candidate. "Stale data from single point of failure" is a recurring class. We have at least three instances (this one + LEARNINGS #113 silent-failure streak + LEARNINGS #110 jq-silent-zero). A Pareto of "stale-data" defects would show ~10% of all lessons. Worth its own PDCA cycle.

Example 6 — LEARNINGS #157 (migration idempotency) — clean PDCA

Already in 10 §3b Example 3. Causal analysis was implicit; fix deployed; no recurrence.

Defect class. data (migration pattern). **Defect prevention principle:** idempotency as a standard pattern for any script that may re-run.

Example 7 — LEARNINGS #88 (scheduler.log probe location) — discovery through investigation

The Frappe scheduler runs but DB connects fail silently (LEARNINGS #87). The probe location for the scheduler was wrong — we were looking at `docker logs` when the canonical location was `sites/<site>/logs/scheduler.log` (LEARNINGS #88).

Defect class. process (monitoring gap). **Lesson.** When looking for a Frappe-internal issue, check Frappe-internal logs first, container logs second.

4. Responsibilities

Role	Responsibilities
Venkat Narasimha (Owner)	Convenes causal-analysis meetings within 48h of SEV-1. Approves LEARNINGS entries before commit (sanity check). Reviews monthly LEARNINGS review. Owns the categorisation enforcement.
Processbricks admin	Drafts LEARNINGS entries from post-mortems. Categorises defects within 7 days. Cross-links lessons into runbooks within 7 days.
Subagents (automation)	Surface findings to LEARNINGS.md via the parent-verify path; do not commit LEARNINGS entries directly (human review required).
All operators	When a subagent diagnosis disagrees with your hypothesis, **verify before dismissing** (LEARNINGS #72 + CapitalCase lesson).

5. Compliance Measurement

Check	Frequency	Owner	Source of truth
Every SEV-1/2 has a post-mortem within 24h	Per incident	PA	post-mortem file mtime
Every post-mortem does ≥ 1 RCA pass	Per incident	PA	post-mortem “Root cause” section
Every post-mortem writes ≥ 1 LEARNINGS entry	Per incident	PA	LEARNINGS.md diff
LEARNINGS entries cited in runbooks within 7d	Per lesson	PA	grep runbook footers
Causal-analysis meeting within 48h of SEV-1	Per SEV-1	VN	meeting notes
Defects categorised within 7 days	Per lesson	PA	LEARNINGS.md tags
Recurrence rate < 5% (per 90-day window)	Per quarter	VN	post-mortem vs LEARNINGS text search
Lessons-citation coverage = 100%	Continuous	PA	grep
Monthly LEARNINGS.md review	Monthly	PA	review notes

KPI dashboard (informal):

KPI	Target	Source
Recurrence rate (same root cause within 90d)	< 5%	post-mortem + LEARNINGS cross-ref
Time-to-detection (deploy → first failure report)	≤ 24h	git log + post-mortem
Time-to-RCA (incident detection → root-cause confirmation)	≤ 30 min	post-mortem timeline
Lessons-citation coverage	100%	runbook footers
Defect categorisation coverage	100% of new lessons	LEARNINGS.md tags
Causal-analysis meeting within 48h SLA	100% of SEV-1	meeting notes

6. Exceptions

1. **SEV-3/4 post-mortems are optional** (07 §3.6.2). Defect prevention still applies: a SEV-3 that reveals a class of issue (e.g., the CapitalCase color fix was discovered as SEV-3) should still produce a LEARNINGS entry.
2. **Causal-analysis meeting for SEV-2 is optional** but recommended when the SEV-2 reveals a process gap.
3. **External finding (e.g., CVE) doesn’t require an internal post-mortem**, but the LEARNINGS entry is still required.

4. **All other exceptions** follow 01-info-security §6.

6a. Edge Cases & Decision Matrix

Edge case 1 — The same defect recurs within 90 days

- **Trigger.** Lesson X was written; a new SEV-1 has the same root cause within 90 days.
- **Decision matrix.**

Action	Why
Treat as SEV-1 regardless of actual severity	YES (07 §6a Edge 3)
Re-run the prior post-mortem template	NO — fresh template; cite prior in "Related"
Add "Reinforced, recurrence" annotation to prior LEARNINGS	YES
Spawn a verification subagent: was the runbook actually updated?	YES — was the fix ever applied?

- **Default action.** Fresh post-mortem with prior cited. Root cause: the prior fix wasn't applied or wasn't effective.

Edge case 2 — A LEARNINGS entry has no runbook citation 7 days post-publication

- **Trigger.** Lesson #N committed; 7 days later, no runbook cites #N.
- **Decision matrix.**

Action	Why
Flag in monthly review	YES
Spawn a subagent to find the right runbook	YES — sometimes the lesson applies to a runbook that doesn't exist yet (catalyst to author one)
Delete the lesson	NO — the finding is real even if we haven't actioned it

- **Default action.** Quarterly review: lessons with no citation 30+ days old are reviewed by Venkat. Decide: cite, fix, or close (with explanation).

Edge case 3 — The RCA points to a fix we can't make (upstream)

- **Trigger.** Root cause is in Frappe or ERPNext core (LEARNINGS #45, #152, #155 — all upstream). We're not allowed to edit core (MEMORY "SOUL NEVER rules").
- **Decision matrix.**

Action	Why
Document the upstream issue	YES
Add a runbook workaround	YES — that's our control
Open an upstream issue (GitHub)	OPTIONAL
Wait for upstream fix	NO — we need the workaround today

- **Default action.** Document + workaround + (optional) upstream issue. The lesson is "we have a workaround for X upstream bug; don't waste time investigating again".

Edge case 4 — The root cause is "operator error"

- **Trigger.** Post-mortem identifies an operator's mistake as the root cause.
- **Decision matrix.**

Action	Why
Blame the operator in the post-mortem	NO — blameless culture (05.2)
Identify the process gap that allowed the mistake	YES — "why did the process allow this?"
Treat as a training issue	OPTIONAL — training is one mitigation, not the only one
Skip the LEARNINGS entry because "they know better now"	NO — the lesson is for future operators too

- **Default action.** Bypass the person; target the gap. If 5 Whys ends at "operator did X", continue: "Why was X possible? Why didn't the runbook prevent X? Why didn't the smoke test catch X?". The actionable root cause is the *system* gap, not the *operator*.

Edge case 5 — The post-mortem finds no root cause

- **Trigger.** Investigation complete; we know what happened but not why.
- **Decision matrix.**

Action	Why
Write the post-mortem with "root cause unknown"	YES — honest
Skip the LEARNINGS entry	NO — the lesson is "we don't understand X"
Schedule a follow-up investigation	YES — within 30 days

- **Default action.** Document honestly. The LEARNINGS entry becomes "Lesson: X is reproducible but root cause unknown; investigation continues". This is rare but valid.

Edge case 6 — The defect is from a vendor's bug

- **Trigger.** A bug in Frappe / ERPNext / HRMS / DuckDNS / Let's Encrypt / OS package causes a SEV.
- **Decision matrix.**

Action	Why
Write the LEARNINGS entry under "external" or "vendor" tag	YES
Add to the dependency-risk register	YES — see 08 §6
Demand a fix from vendor	OPTIONAL — depends on impact
Document a workaround	YES

- **Default action.** LEARNINGS entry + workaround + dependency-risk note. Don't expect vendor SLA in v1.

Edge case 7 — Two SEV-1s share a contributing factor but different root causes

- **Trigger.** SEV-1a and SEV-1b within 30 days; both involve "stale data from heartbeat"; root causes differ.
- **Decision matrix.** Both LEARNINGS entries get written. A third LEARNINGS entry (or a meta-lesson) captures the contributing factor. The PDCA cycle would target the contributing factor.
- **Default action.** Two post-mortems + one PDCA cycle on the common factor.

Edge case 8 — The fix for one defect enables another

- **Trigger.** Adding BACKUP_0K sentinel fixed silent failures (LEARNINGS #113/#114) but the sentinel pattern was later found to be missing from another script that silently failed.
- **Decision matrix.** Two LEARNINGS entries (the original + the sibling). The pattern is "sentinel lines are a class of fix; audit all scripts for sentinel lines after any new fix lands".
- **Default action.** Two entries + a meta-cycle on "audit all scripts when a fix lands".

7. Related Documents

- 09-cmm-maturity-assessment.md — HH-CMM-01 — Defect prevention feeds the maturity score.
- 10-process-improvement.md — HH-CMM-02 — PDCA; defect prevention is the Act step.
- 11-quantitative-management.md — HH-CMM-03 — Defect prevention metrics come from QPM.
- 07-incident-management.md — SEV ladder; feeds the post-mortem requirement.
- 08-business-continuity.md — DR-related defects and DR-drill findings.
- 05-process/05.2-post-mortem.md — Post-mortem template; the canonical place where RCA happens.
- 05-process/05.1-change-management.md — Approval flow for defect-prevention fixes.
- 04-runbooks/ — Where lessons land as runbook updates.
- ../../../../.learnings/LEARNINGS.md — The repository.
- ../../../../MEMORY.md — Tech stack + standing rules.

8. Revision History

Version	Date	Author	Changes
1.0	2026-08-29	venkat-narasimha	Initial

9. Implementation Checklist

Concrete actions derived from this policy. Owner initials: VN = Venkat Narasimha; PA = Processbricks admin. Status as of 2026-08-29.

Immediate (this week)

- ☐ **Hold the first formal causal-analysis meeting** for a recent SEV-1/2 (suggested: re-review the 2026-08-29 outage retroactively to validate the structure). Owner: VN. Target: 2026-09-05. Status: Not Started.
- ☐ **Add Fishbone template** to ../05-process/05.2-post-mortem.md §"Part 1". Owner: VN. Target: 2026-09-05. Status: Not Started.
- ☐ **Add 5 Whys template** to ../05-process/05.2-post-mortem.md §"Part 1". Owner: VN. Target: 2026-09-05. Status: Not Started.

- ☐ **Cross-link this doc from LEARNINGS.md** footer (so future lesson writers see the categorisation convention). Owner: PA. Target: 2026-09-05. Status: Not Started.

Short-term (2026-Q3)

- ☐ **Tag the 20 most recent LEARNINGS entries** with type + severity + detection-point (per §3.5). Owner: PA. Target: 2026-09-30. Status: Not Started.
- ☐ **First monthly Pareto review** — which defect class dominates? Owner: VN. Target: 2026-09-30. Status: Not Started.
- ☐ **First recurrence-rate measurement** — count SEV-1/2 root causes that match a prior lesson within 90 days. Owner: PA. Target: 2026-09-30. Status: Not Started.
- ☐ **Audit all runbook footers** vs LEARNINGS.md; flag uncited lessons. Owner: PA. Target: 2026-09-30. Status: Not Started.

Medium-term (2026-Q4)

- ☐ **Near-miss log convention** — prefix `#NM-` in LEARNINGS.md for caught-before-incident findings (e.g., LEARNINGS #72 in action). Owner: VN. Target: 2026-12-31. Status: Not Started.
- ☐ **Wire defect prevention metrics into 11** (M9 = recurrence rate, M10 = defect density). Owner: VN. Target: 2026-12-31. Status: Not Started.
- ☐ **Quarterly review of "lessons-cited coverage"** — drive to 100%. Owner: PA. Target: 2026-12-31. Status: Not Started.
- ☐ **At least 4 causal-analysis meetings** held in 2026-Q4. Owner: VN. Target: 2026-12-31. Status: Not Started.

Long-term (2027+)

- ☐ **External RCAR review** — annual review by an external party to challenge our blind spots. Owner: VN. Target: TBD. Status: Not Started.
- ☐ **Defect-density dashboard** — chart defect rate over time. Owner: VN. Target: TBD. Status: Not Started.
- ☐ **Predictive defect prevention** — flag changes likely to cause defects (e.g., deploys to long-stable code paths). Owner: VN. Target: TBD. Status: Not Started.

Recurring verification (runs forever)

- ☐ **Every SEV-1/2 has a post-mortem within 24h** (07 §3.6). Owner: PA. Frequency: per incident. Status: Done.
- ☐ **Every post-mortem writes ≥ 1 LEARNINGS entry**. Owner: PA. Frequency: per incident. Status: Done.
- ☐ **LEARNINGS.md monthly review** — accuracy, cross-refs, tags. Owner: PA. Frequency: monthly. Status: Done.
- ☐ **Lessons-cited coverage quarterly audit**. Owner: PA. Frequency: quarterly. Status: Not Started.
- ☐ **Quarterly Pareto review** — defect-class distribution. Owner: VN. Frequency: quarterly. Status: Not Started.
- ☐ **Annual policy review** (this doc). Owner: VN. Frequency: annually. Status: Done (this revision).

Listen to the subagent's diagnosis — verify before dismissing. The CapitalCase color fix is the lesson. The right root cause is one question away. Document or repeat.